

Reuse Intelligence Audit — Phase 2

Date: 2026-06-26

Scope: Make **Repo Intelligence** own Reuse Intelligence (detect & catalogue reusable repository assets) and turn **Script Generation** into a pure consumer that reuses before generating. Builds directly on Phase 1 (PR #170), which moved folder/naming conventions into the `ProjectConventionProfile`.

Hard constraint: No regressions. Existing generation output must remain **byte-for-byte identical** unless an existing reusable asset is detected. Reuse Intelligence is a pure Repo Intelligence capability; Script Generation must remain a consumer only.

1. What Repo Intelligence already extracts (always, no feature flag)

The repository scan already produces a rich `RepositoryProfile` (`src/context/types.ts`) during a normal scan — **no flag required**. The reuse-relevant fields are already there:

Field	Type	Carries
<code>pageObjects</code>	<code>ClassInfo[]</code>	name, filePath, baseClass, methods (signatures), properties (with selector + locatorType), category
<code>helperFunctions</code>	<code>FunctionSignature[]</code>	name, filePath, params, returnType, jsdoc, category
<code>fixtures</code>	<code>FunctionSignature[]</code>	name, filePath, ...
<code>customCommands</code>	<code>FunctionSignature[]</code>	name, filePath, ...
<code>sharedConstants</code>	<code>{name,value,filePath}[]</code>	shared constants
<code>dataFiles</code>	<code>{name,path,type,recordCount}[]</code>	json / ts / js / csv test-data files
<code>hasApiLayer</code> , <code>hasCustomFixtures</code>	<code>boolean</code>	capability flags

Conclusion: the raw material for a reuse catalogue is already extracted and cached. We do **not** need new scanning, new DB tables, or new feature flags. The catalogue is a **pure derived view** of `RepositoryProfile` — exactly the Phase 1 pattern.

2. Where Script Generation creates assets (and the reuse gaps)

Asset	Generated in	Reuse today	Gap
Page objects	<code>generatePomFiles</code> , <code>generatePageObject</code> , <code>matchPageObjects</code>	✅ <code>findExistingPageObject</code> / <code>existingCoversAllLocators</code> / <code>selector reuse</code> / <code>matchPageObjects</code>	Detection reads <code>config.repoProfile</code> directly — Script Gen reaching into raw repo data instead of asking Repo Intelligence.
Helpers	(AI prompt path via <code>TrueReuseEngine</code>)	⚠️ Only the flagged, DB-backed <code>TrueReuseEngine</code> feeds the prompt. Deterministic path: none.	No catalogue of helpers consumed by the deterministic path; no name-based “does AuthHelper exist?” API.
Fixtures	<code>generatePomFiles</code> (<code>test-fixtures.ts</code> , <code>auth.ts</code>)	⚠️ Boolean <code>analysis.hasFixtures</code> suppresses scaffold	No named fixture catalogue → can't reuse <code>baseFixture</code> / <code>authFixture</code> by name.
Test data	<code>buildTestDataModule</code> → <code>test-data.ts</code>	❌ none	Never asks whether the repo already has <code>users.json</code> / <code>test-data.ts</code> / <code>builders</code> .
API clients	—	❌ not catalogued	<code>UserApi</code> / <code>OrderApi</code> not surfaced.
Components	—	❌ not catalogued	<code>HeaderComponent</code> / <code>MenuComponent</code> not surfaced.

Existing reuse infrastructure (keep, don't break)

- `findExistingPageObject` + `normalizePageObjectName` + `matchExistingLocator` + `existingCoversAllLocators` + `selectorCore` — sophisticated selector-level page-object reuse. **Preserve heuristics exactly.**
- `matchPageObjects` — test-case path: maps login/inventory/cart/checkout to existing POs and reuses high-level methods (`login()` , etc.).
- `TrueReuseEngine` (`src/services/true-reuse-engine.ts`) — DB/method-index, `ENABLE_TRUE_REUSE` (default off), prompt-only.

3. Design — the Reuse Catalogue (single source of truth)

Extend `ProjectConventionProfile` with a `reuse` catalogue, built **purely** from `RepositoryProfile` inside `buildConventionProfile()`:

```
{
  "folders": { ... },           // Phase 1
  "reuse": {
    "pageObjects": [ { "name": "LoginPage", "path": "pages/LoginPage.ts",
                      "methods": ["login", "logout"], "locators": ["username", "pass-
word"],
                      "baseClass": "BasePage" } ],
    "helpers": [ { "name": "AuthHelper", "path": "utils/AuthHelper.ts",
                  "functions": ["login", "logout"] } ],
    "fixtures": [ { "name": "baseFixture", "path": "fixtures/baseFixture.ts" } ],
    "apis": [ { "name": "UserApi", "path": "api/UserApi.ts" } ],
    "components": [ { "name": "HeaderComponent", "path": "components/Header.ts" } ],
    "testData": [ { "name": "users.json", "path": "tests/data/users.json", "type": "
json" } ]
  }
}
```

Page-object entries also carry the full `ClassInfo (raw)` so the existing selector-level matcher keeps working unchanged.

“Ask Repo Intelligence” query APIs (for every future consumer)

- `findReusablePageObject(conv, name)` → reuse the repo’s `LoginPage` instead of generating one.
- `findReusableHelper(conv, name)` → reuse `AuthHelper.login()`.
- `findReusableFixture(conv, name)` → reuse `baseFixture`.
- `findReusableTestData(conv, nameOrFile)` → reuse `checkout_data.json`.
- `findReusableApi`, `findReusableComponent`, `hasReusableAssets(conv)`.

These are reusable by **Healing, Repo Patch, Migration, PR Generator, Learning Engine, Framework**

Conversion, Component Intelligence — not just Script Generation.

apis / components derivation

`RepositoryProfile` has no dedicated api/component lists. We derive them **honestly** from already catalogued names: classes/helpers matching `/(Api|Client|Service|Endpoint)$/i` → `apis`; matching `/Component$/i` → `components`. Empty arrays when nothing matches (as in the spec example).

4. Script Generation becomes a consumer

1. **Page objects:** route `findExistingPageObject` and `matchPageObjects` to source their candidate list from `conv.reuse.pageObjects` (which mirrors the same `RepositoryProfile` data) instead of reading `config.repoProfile.pageObjects` directly. **Identical data ⇒ identical output**, but Script Gen now asks Repo Intelligence.
2. **Catalogue surfaced** on the convention profile + `RepoStructureAnalysis.conventions` so the prompt path and future consumers can ask “does X already exist?”.

Zero-regression guarantee

- Greenfield (no `repoProfile`) ⇒ **empty catalogue** ⇒ no behavior change.
- Connected repos: the catalogue is a faithful mirror of the same cached profile data the engine already reads, so routing existing lookups through it yields identical results.

Explicitly out of scope (to honor “no regressions”)

- Rewiring test-data emission to drop the generated `test-data.ts` in favor of a repo file (risk: the repo file may not contain the records the spec binds to). Test data is **catalogued and queryable** but emission is unchanged — same posture Phase 1 took on data-format/alias.
- No new feature flags, scanning, or DB tables.

5. Validation plan

- `npx tsc --noEmit clean`.
- New `tests/unit/reuse-intelligence.test.ts` : greenfield ⇒ empty catalogue; connected repo ⇒ catalogue populated; query APIs resolve by normalized name; apis/components derived honestly.
- `repo-analyzer` / `repo-pattern-analyzer` / `project-convention-profile` suites stay green.
- Page-object reuse parity confirmed (same matches before/after routing through the catalogue).